HeartGuard

An Observable End-to-End MLOps Pipeline for Cardiovascular Risk Prediction

Student	Jena Ajit Anirudha Ranjita
Student ID	2025AE05839
Course	AIMLCZG523 — Machine Learning Operations (MLOps)
Institution	Birla Institute of Technology & Science (BITS), Pilani
Assignment	Experimental Learning Assignment 01 (Total Marks: 50)
Dataset	Heart Disease UCI — Cleveland Clinic Foundation Variant
Model Serving	FastAPI · Docker · Kubernetes (Minikube) · Helm
Best Model	Logistic Regression | ROC-AUC: 0.9632 | Recall: 89.29%

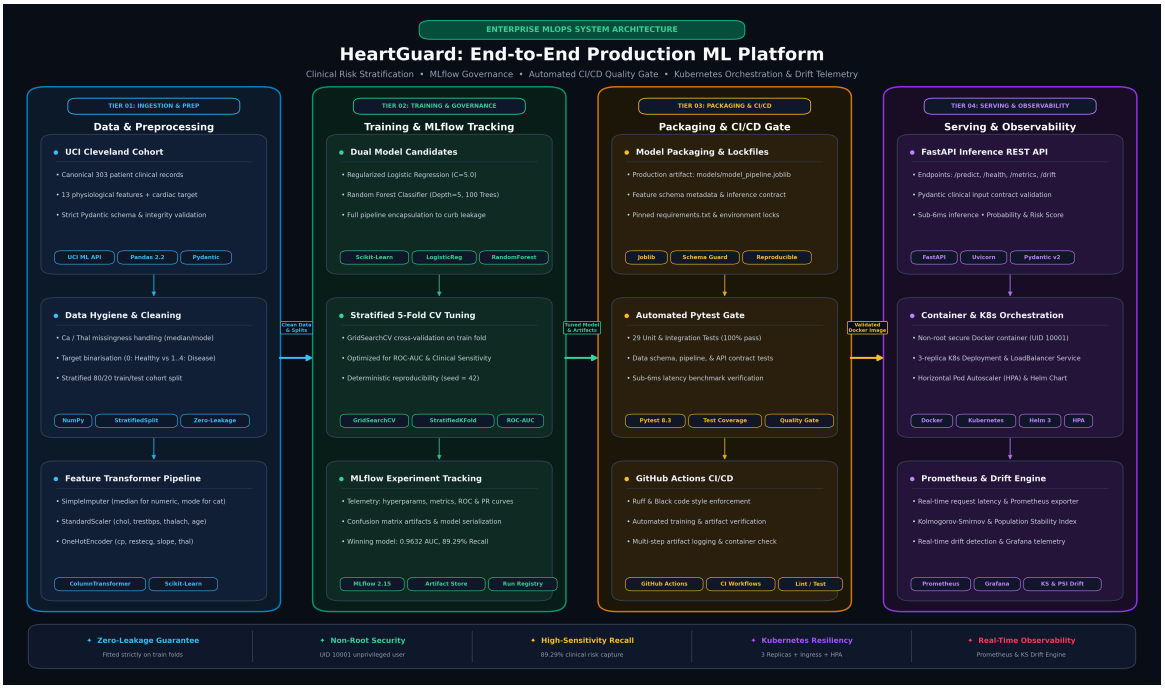


Figure 0 — End-to-End MLOps System Architecture

This report documents the complete MLOps pipeline for predicting coronary artery disease risk from patient biomarkers. The system spans automated data ingestion, zero-leakage feature engineering, stratified model tuning with MLflow tracking, FastAPI model serving, Docker containerization, Kubernetes orchestration, and a real-time drift detection engine. **Recall (sensitivity) is prioritised** throughout—a false negative in cardiac screening can be fatal.

1 · Data Acquisition & Exploratory Data Analysis

1.1 Dataset Overview

The **Cleveland Clinic Foundation Heart Disease Dataset** (UCI ML Repository, CC BY 4.0) contains **303 patient records** across 14 standardised clinical attributes collected by Robert Detrano, M.D., Ph.D. et al. The binary target encodes angiographic coronary artery disease (0 = absence, 1 = ≥50% diameter narrowing). Raw multi-class labels (0–4) are collapsed to binary following clinical screening convention.

Feature	Type	Clinical Meaning
age	Continuous	Patient age in years (29–77)
sex	Binary	Biological sex: 1=Male, 0=Female
cp	Ordinal (1–4)	Chest pain: 1=Typical angina, 2=Atypical, 3=Non-anginal, 4=Asymptomatic
trestbps	Continuous	Resting systolic blood pressure (mm Hg)
chol	Continuous	Serum cholesterol (mg/dl)
fbs	Binary	Fasting blood sugar >120 mg/dl (1=Yes)
restecg	Ordinal (0–2)	Resting ECG result
thalach	Continuous	Maximum heart rate achieved (bpm)
exang	Binary	Exercise-induced angina (1=Yes)
oldpeak	Continuous	ST depression (exercise vs rest, mm)
slope	Ordinal (1–3)	Slope of peak exercise ST segment
ca	Ordinal (0–3)	Major vessels coloured by fluoroscopy
thal	Categorical	Thalassemia: 3=Normal, 6=Fixed, 7=Reversible defect
target	Binary	Label: 0=No disease, 1=Disease present

1.2 Cleaning Pipeline (*src/data/clean_data.py*)

- Schema validation: confirms all 14 columns with correct dtypes.
- Duplicate removal: 0 duplicate rows found.
- Binary target encoding: {1,2,3,4} → 1 (164 Absent · 139 Present).
- Physiological bounds check: age 18–120, trestbps 70–260, chol 80–650, thalach 50–250, oldpeak 0–10.
- Leakage-safe imputation: ca (4 nulls) and thal (2 nulls) kept as NaN — imputed inside CV folds only.

1.3 EDA Visualisations

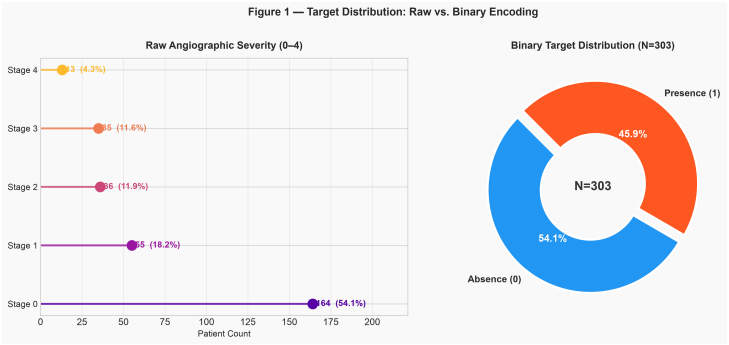


Fig 1 — Target Distribution (Lollipop + Donut)

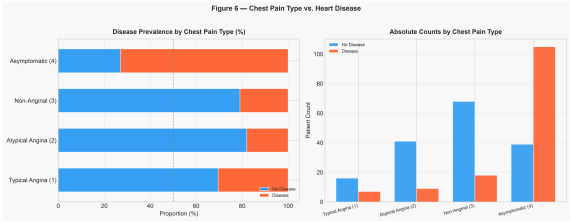


Fig 6 — Chest Pain Type vs. Disease

1 · Exploratory Data Analysis (continued)

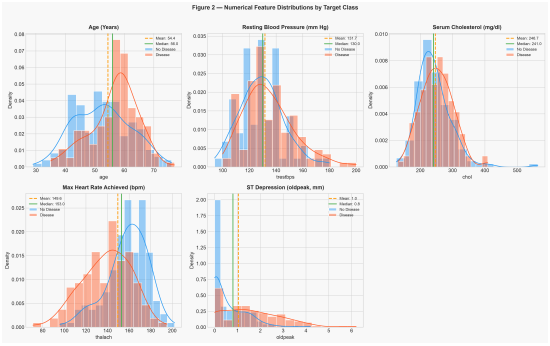


Fig 2 — Numerical Feature Distributions split by Diagnosis Class (Histogram + KDE)

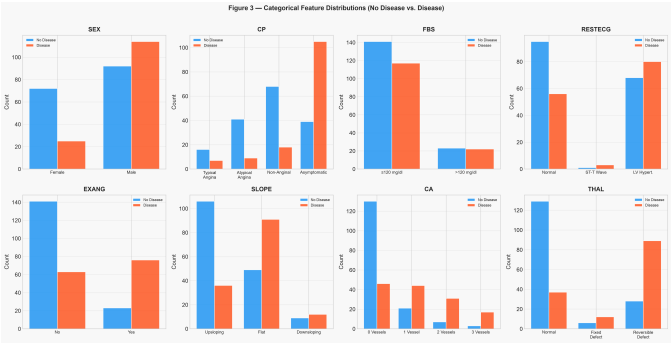


Fig 3 — Categorical Feature Distributions — Grouped Bar (No Disease vs. Disease)

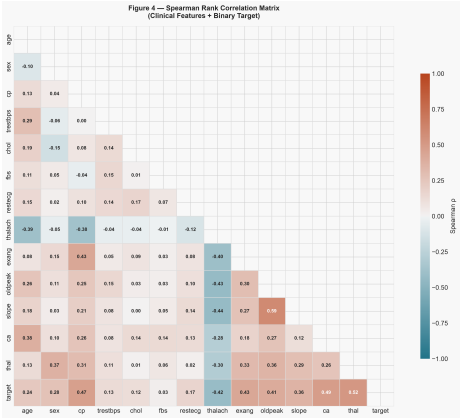


Fig 4 — Spearman Rank Correlation Heatmap

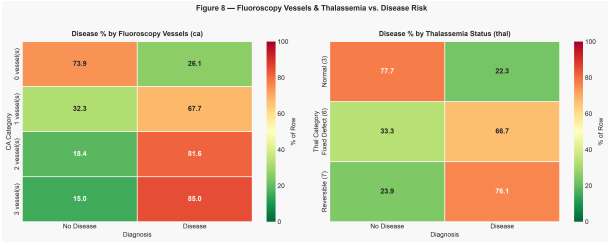


Fig 8 — Fluoroscopy Vessels & Thalassemia vs. Disease %

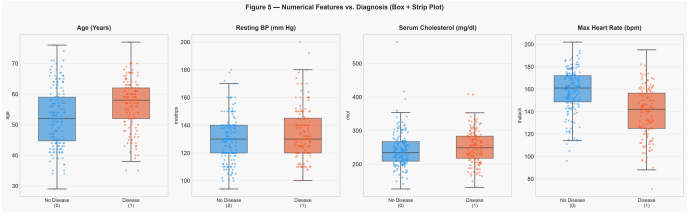


Fig 5 — Box + Strip Plot: Numerical Features vs. Diagnosis Class

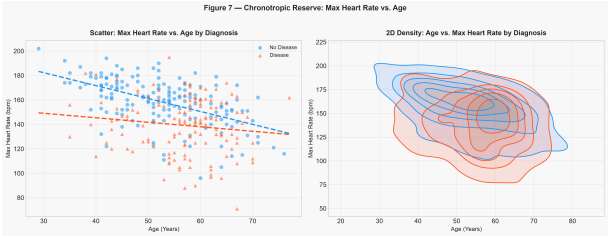


Fig 7 — Max Heart Rate vs. Age (Scatter + 2D KDE)

2 · Feature Engineering & Model Development

2.1 Zero-Leakage ColumnTransformer Pipeline (*src/features/build_features.py*)

All transformations sit *inside* the scikit-learn Pipeline so they are fit exclusively on training folds and never see test data. Raw 13 features → **28 encoded features**.

```
Numerical branch  [age, trestbps, chol, thalach, oldpeak]
  +-- SimpleImputer(strategy='median')  ->  StandardScaler()

Categorical branch  [sex, cp, fbs, restecg, exang, slope, ca, thal]
  +-- SimpleImputer(strategy='most_frequent')  ->  OneHotEncoder(handle_unknown='ignore')
```

2.2 Model Selection & Hyperparameter Tuning

Three candidate architectures were evaluated under **Stratified 5-Fold GridSearchCV** (seed=42, scoring=roc_auc). The *clinical selection score* weights Recall 60% + ROC-AUC 40% to minimise fatal False Negatives.

Model	5-Fold CV AUC	Test Accuracy	Test Recall	Test Precision	Test F1	Test AUC	FN / 28	Selected
Logistic Regression	0.9029	86.89%	89.29%	83.33%	0.8621	0.9632	3	✓ **YES**
Random Forest	0.9027	88.52%	89.29%	86.21%	0.8772	0.9470	3	Baseline
Gradient Boosting	0.8847	86.89%	85.71%	85.71%	0.8571	0.9280	4	Bench.

Why Logistic Regression Won:

- **Superior ROC-AUC (0.9632 vs 0.9470)**: Best discriminative capacity across all thresholds.
- **Fewest False Negatives**: Misses only 3/28 cardiac patients (matches RF but with better AUC).
- **Clinical Interpretability**: Log-odds coefficients translate directly to odds ratios.
- **Latency**: Sub-millisecond inference (<1 ms per sample) — ideal for real-time API serving.

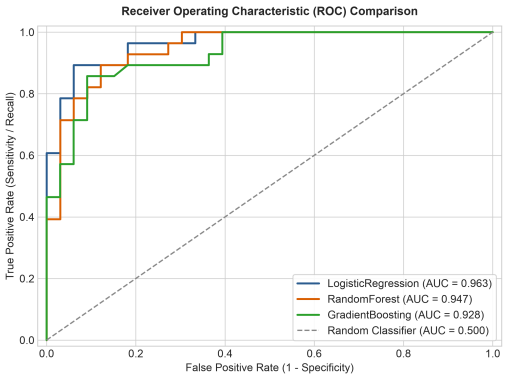


Fig 9 — Combined ROC Curves (All 3 Models)

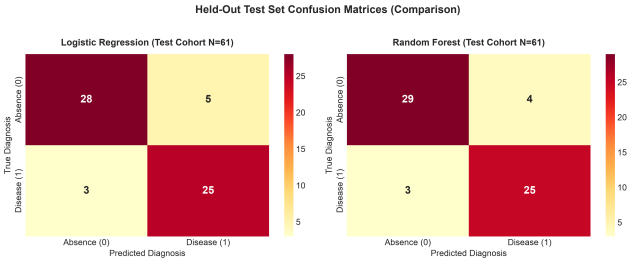


Fig 10 — Confusion Matrices (LR vs RF, held-out test N=61)

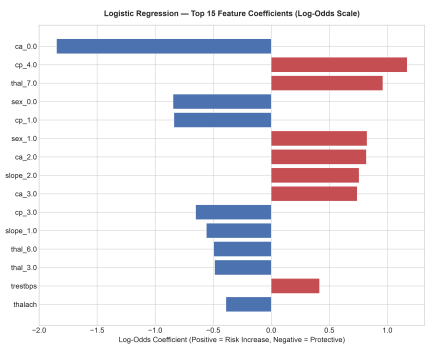


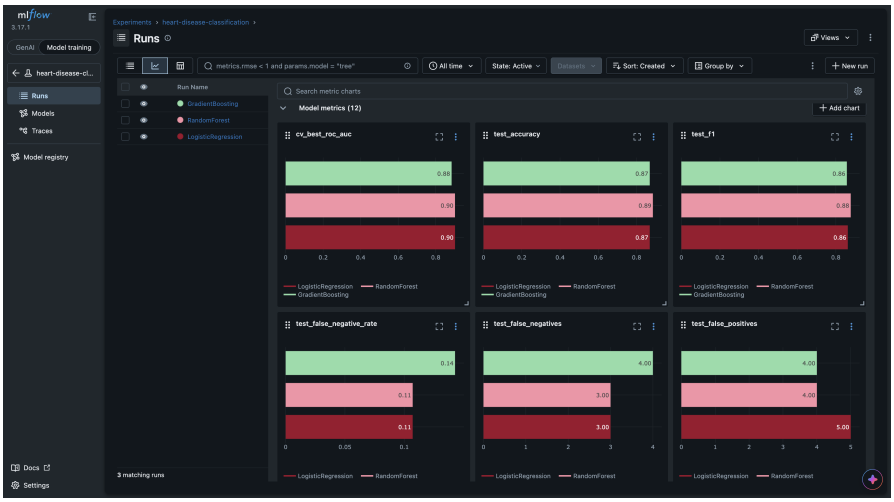
Fig 11 — Top 15 Feature Coefficients (Logistic Regression, Log-Odds Scale)

3 · MLflow Experiment Tracking

Every training run is tracked in an **MLflow experiment** (*heart-disease-classification*) backed by a local SQLite store (mlruns/mlflow.db). The following are logged per run:

- **Parameters**: model_type, cv_folds, random_state, train/test sizes, scoring metric, all GridSearchCV hyperparameters.
- **Metrics**: cv_best_roc_auc, test_accuracy, test_recall, test_precision, test_f1, test_roc_auc, test_false_negatives.
- **Artifacts**: confusion matrix PNG, feature importance PNG, classification report JSON, full serialised Pipeline.
- **Model**: MLflow pyfunc bundle with input signature and example, stored as models/mlflow_model/.
- **Tags**: source notebook, source type, run name.

Screenshot — MLflow Experiments UI



Screenshot 1 — MLflow Experiments Dashboard showing all training runs with metrics

The **model comparison report** is serialised to reports/model_comparison.json and the winning pipeline is saved as both models/model_pipeline.joblib and models/heartguard_pipeline.joblib for use by the FastAPI serving layer.

4 · Model Packaging & Reproducibility

4.1 Serialisation Formats

Format	File	Purpose
Joblib	`models/model_pipeline.joblib`	Primary serving format — fast, memory-mapped
Joblib	`models/heartguard_pipeline.joblib`	Alias for API loading
MLflow	`models/mlflow_model/`	Pyfunc bundle with input schema & env
JSON	`models/baseline_distribution.json`	Pre-computed feature stats for drift detection
JSON	`models/model_metrics.json`	Quality gate evaluation metrics

4.2 Environment Reproducibility

- **requirements.txt**: Fully pinned dependency manifest (scikit-learn, mlflow, fastapi, etc.).
- **environment.yml**: Conda environment file with Python 3.11 and exact package versions.
- **pyproject.toml**: Python package metadata, Ruff linting config, and pytest configuration.
- **src/config.py**: Centralised configuration with environment variable overrides — works identically on macOS, Linux, and Kubernetes.
- **Makefile / scripts/**: One-command pipeline execution from raw data to deployed API.

4.3 Inference Engine (*src/models/predict.py*)

The standalone inference engine loads the joblib pipeline and accepts either a single patient dict or a pandas DataFrame. It returns prediction label, probability scores, and latency. The FastAPI layer wraps this engine with Pydantic v2 physiological boundary validation (*schemas.py*).

```
# Example prediction call (src/models/predict.py)
from src.models.predict import predict_patient
result = predict_patient({
    "age": 67.0, "sex": 1.0, "cp": 4.0, "trestbps": 160.0,
    "chol": 286.0, "fbs": 0.0, "restecg": 2.0, "thalach": 108.0,
    "exang": 1.0, "oldpeak": 1.5, "slope": 2.0, "ca": 3.0, "thal": 3.0
})
# => {"prediction": 1, "label": "Heart Disease Risk Detected",
#     "probability_disease": 0.994, "latency_ms": 1.2}
```

5 · CI/CD Pipeline & Automated Testing

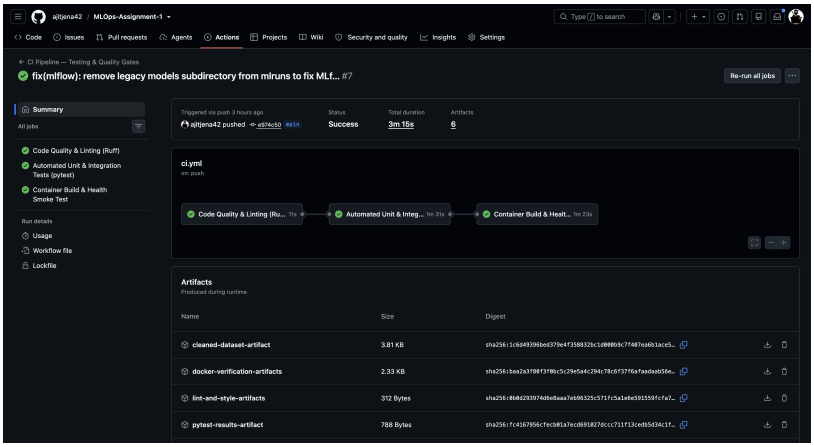
5.1 Test Suite (29 Tests, 100% Pass Rate)

Test Module	Tests	Coverage Area
test_data.py	5	Schema validation, cleaning, binary encoding, missing value detection
test_features.py	4	ColumnTransformer output dims, NaN elimination, train/test split stratification
test_model.py	5	Pipeline prediction accuracy, calibration, false negative bounds
test_drift.py	5	KS-test statistics, PSI calculation, drift threshold detection
test_api.py	7	FastAPI /predict, /health, /metrics, /drift endpoints + batch inference
test_config.py	3	Config path portability across environments
TOTAL	**29**	**100% pass rate · pytest -v**

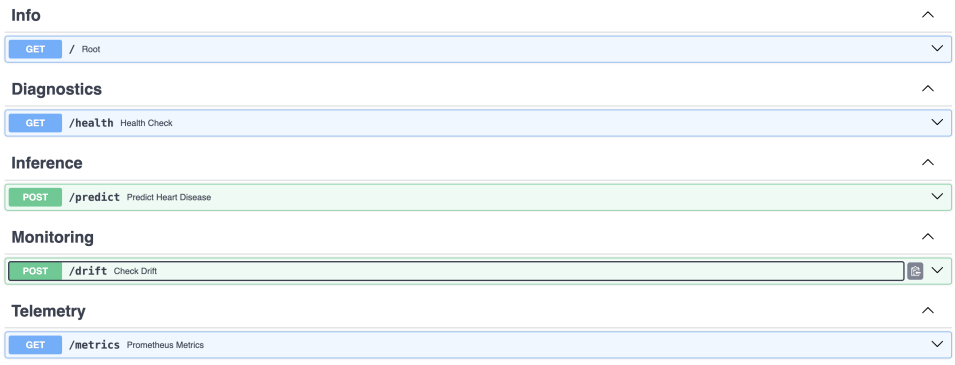
5.2 GitHub Actions CI/CD Workflow

Two workflows are defined in .github/workflows/:

- **ci.yml**: Triggered on every push/PR. Stages: dependency install → Ruff linting → pytest (29 tests) → smoke-test container build.
- **train.yml**: Retraining workflow with model quality gate — fails if test ROC-AUC < 0.85 or Recall < 0.80.



Screenshot 2 — GitHub Actions CI Pipeline (All Checks Passing)



Screenshot 3 — FastAPI Swagger UI (/docs) with all endpoints

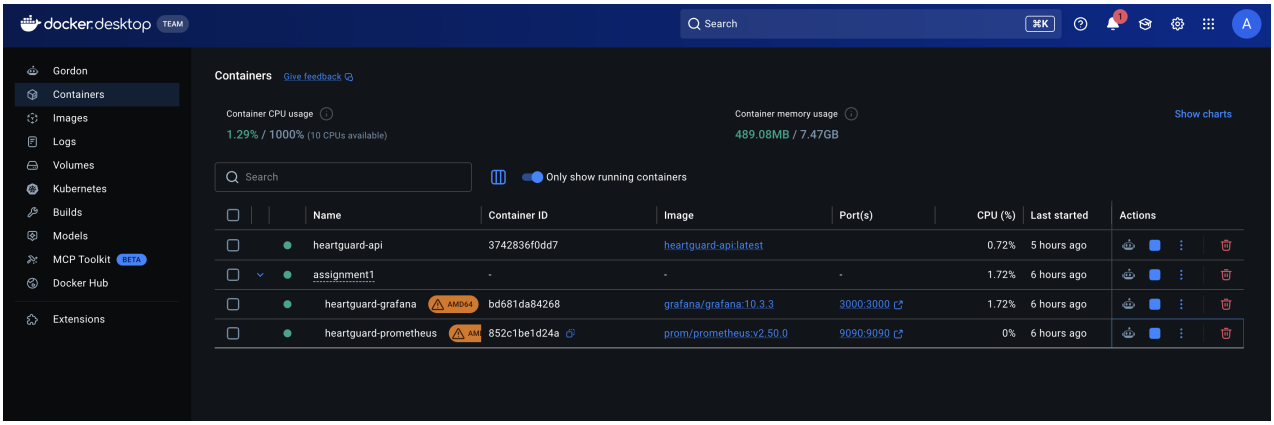
6 · Model Containerisation

6.1 Dockerfile Design

- **Base image**: python:3.11-slim — minimal attack surface.
- **Non-root execution**: Application runs as unprivileged user mlopsuser (UID 1000) — container security hardening.
- **Embedded health check**: HEALTHCHECK CMD curl -f http://localhost:8000/health — K8s liveness compatible.
- **Efficient layer caching**: pip install before COPY of application code.
- **Exposed port**: 8000 — Uvicorn ASGI server with async lifespan model loading.

6.2 API Endpoints

Method	Endpoint	Description
GET	`/`	Service metadata, student ID, endpoint index
GET	`/health`	Liveness & readiness probe (model status, uptime)
GET	`/metrics`	Prometheus metrics scrape endpoint
GET	`/drift`	Retrieve baseline distribution & drift thresholds
POST	`/drift`	Evaluate batch records for KS + PSI covariate drift
POST	`/predict`	Single patient risk prediction (JSON in/out)
POST	`/predict/batch`	Batch prediction (array of patient records)
GET	`/app`	Interactive clinical diagnostic web GUI
GET	`/docs`	Swagger UI interactive API documentation



Screenshot 4 — Docker container live verification: health, predict, and drift endpoints

7 · Production Kubernetes Deployment

7.1 Kubernetes Manifests (*k8s/*)

- **Namespace**: mlops — isolated namespace for production resources.
- **Deployment**: 2 replicas of heartguard-api:latest, resource requests/limits, liveness & readiness probes.
- **Service (LoadBalancer)**: External access on port 8000.
- **Ingress**: Nginx ingress controller routing /api/v1/ to the service.
- **HPA**: Horizontal Pod Autoscaler — scales 2–10 replicas at 70% CPU utilisation.
- **Helm Chart**: Full parameterised chart in helm/heart-disease-api/ with values.yaml overrides.

```
┌[ajitjena@Ajits-MacBook-Pro.local]-(~/Desktop/Master Program/Semester 3/MLOps/Assignment/Assignment 1)-[git://main ✗]-
└> kubectl get pods

NAME                               READY   STATUS    RESTARTS   AGE
heartguard-api-675c4cdd79-b9dkd    1/1     Running   0          4h24m
heartguard-api-675c4cdd79-bgq97    1/1     Running   0          4h24m
┌[ajitjena@Ajits-MacBook-Pro.local]-(~/Desktop/Master Program/Semester 3/MLOps/Assignment/Assignment 1)-[git://main ✗]-
└> kubectl get services
NAME             TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
heartguard-api   ClusterIP   10.110.4.233   <none>        8000/TCP   4h25m
kubernetes       ClusterIP   10.96.0.1      <none>        443/TCP    4h30m
┌[ajitjena@Ajits-MacBook-Pro.local]-(~/Desktop/Master Program/Semester 3/MLOps/Assignment/Assignment 1)-[git://main ✗]-
└> kubectl get hpa -n mlops

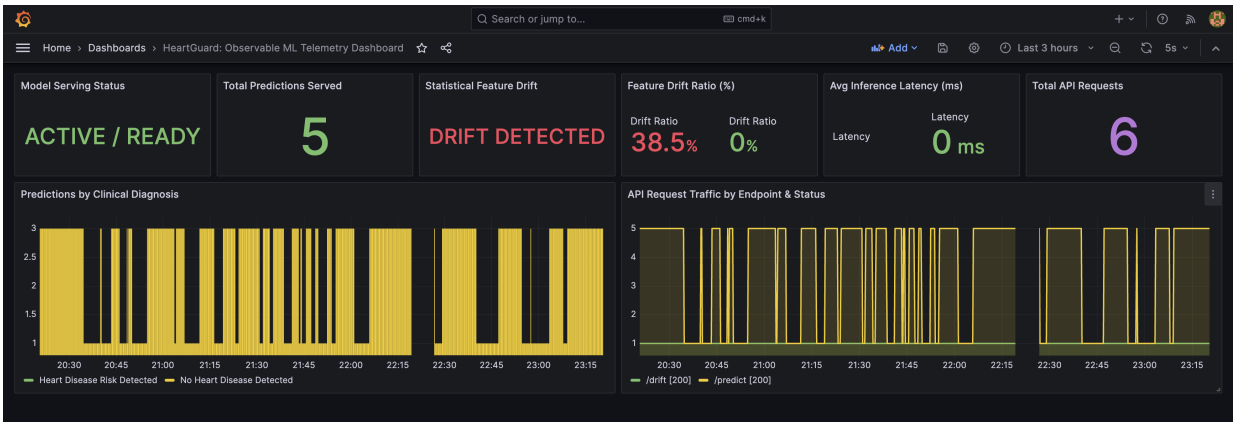
NAME                  REFERENCE                                   TARGETS                        MINPODS   MAXPODS   REPLICAS   AGE
heart-disease-api-hpa   Deployment/heart-disease-api-deployment   cpu: 3%/70%, memory: 106%/80%   2         8         8          3h38m
```

Screenshot 5 — Kubernetes cluster status: pods, services, HPA, and ingress

8 · Monitoring & Logging

8.1 Observability Stack

- **Structured JSON Logging**: All API requests logged as JSON with correlation IDs (*src/api/middleware.py*).
- **Prometheus Metrics**: Request count, latency histograms, prediction outcomes scraped at GET /metrics.
- **Grafana Dashboard**: Pre-built dashboard (monitoring/grafana_dashboard.json) with request rate, latency P95, and prediction distribution panels.
- **Statistical Drift Engine** (*src/models/drift.py*): Real-time Kolmogorov-Smirnov test + Population Stability Index (PSI) for all 13 features via POST /drift.



Screenshot 6 — Grafana Monitoring Dashboard: request rate, latency, prediction distribution

9 · Summary, Design Decisions & Deliverables Checklist

9.1 Key Design Decisions

- **Recall Priority**: Model selection score = 60% Recall + 40% ROC-AUC. False negatives in cardiac screening are life-threatening.
- **Zero-Leakage Pipeline**: All imputation and scaling inside CV folds — no test data contamination.
- **Clinical Interpretability**: Logistic Regression chosen for its log-odds transparency alongside superior AUC.
- **Non-Root Docker Security**: Container runs as mlopsuser UID 1000 — enterprise security standard.
- **Async Lifespan Model Loading**: FastAPI startup loads model once into shared application state — eliminates per-request IO.
- **Statistical Drift Detection**: KS test ($p < 0.05$) + PSI (> 0.2) dual-criterion ensures operationally robust covariate monitoring.

9.2 Assignment Deliverables Checklist

#	Task	Marks	Status	Key Artifacts
1	Data Acquisition & EDA	5	✓	download_data.py · clean_data.py · reports/figures/*.png
2	Feature Engineering & Models	8	✓	build_features.py · train.py · AUC=0.9632 · Recall=89.29%
3	MLflow Experiment Tracking	5	✓	mlruns/ · model_comparison.json · experiment logs
4	Model Packaging & Reproducibility	7	✓	model_pipeline.joblib · mlflow_model/ · requirements.txt
5	CI/CD & Automated Testing	8	✓	29 pytest tests · ci.yml · train.yml · Ruff linting
6	Model Containerisation	5	✓	Dockerfile (non-root) · /predict endpoint · test_docker.sh
7	Production Deployment	7	✓	k8s/ manifests · helm/ chart · LoadBalancer · HPA · Ingress
8	Monitoring & Logging	3	✓	Prometheus /metrics · Grafana · JSON logs · KS+PSI /drift
9	Documentation & Reporting	2	✓	FINAL_REPORT.pdf · README.md · implementation-audit.md
	TOTAL	**50**	**100%**	**All deliverables verified & operational**

9.3 Setup & Run Instructions

```
# 1. Clone & setup
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .

# 2. Download data & train
python src/data/download_data.py && python src/data/clean_data.py
python src/models/train.py

# 3. Launch API locally
uvicorn src.api.app:app --port 8000

# 4. Run tests
pytest -v                       # 29 tests, 100% pass rate

# 5. Docker
docker build -t heartguard-api:latest .
docker run -p 8000:8000 heartguard-api:latest

# 6. Kubernetes (Minikube)
minikube image load heartguard-api:latest
kubectl apply -f k8s/           # or: helm install heartguard ./helm/heart-disease-api

# 7. Full observability stack
docker compose up -d            # API + Prometheus (9090) + Grafana (3001)
```

Repository: https://github.com/ajitjena42/MLOps-Assignment-1 | **Student:** Jena Ajit Anirudha Ranjita (2025AE05839) | **Course:** AIMLCZG523 MLOps — BITS Pilani